Task 1 — Web App QA & Debug Report

Candidate: Jagadeesh Dangeti | Automation & QA Developer — Fresher

Target: https://demo.realworld.io

Status: Investigation draft. The scenarios below must be executed before being reported as confirmed defects. No live test result is fabricated.

1. Scope and approach

The assessment asks for sign-up, login, create, edit, delete, and logout checks, at least five issues, and one root-cause analysis. This plan covers validation, error handling, authorization, and session behavior. Add the actual browser and test date to the execution log.

2. Candidate issues to investigate — not confirmed

Area	Steps to reproduce	Expected behavior / actual to record	Severity if reproduced	Possible cause to investigate
Sign-up validation	Submit empty fields; try malformed email.	Expected: clear validation; record actual response.	Medium	Missing or inconsistent client/server validation.
Login errors	Submit invalid credentials and blank fields.	Expected: safe, clear error; record actual response.	Medium	API error not mapped to UI or missing form validation.
Article validation	Submit new article with blank title/body.	Expected: invalid content blocked; record actual response.	Medium	Required-field validation missing.
Edit/delete authorization	Try editing/deleting own and another user's article if possible.	Expected: only owner can modify; record actual response.	High	Authorization enforced only in UI or absent server-side.
Logout/session	Log in, refresh, log out, press Back, revisit protected page.	Expected: protected actions require valid session; record actual response.	High	Token cleanup or route guard inconsistency.

3. Root-cause analysis — logout/session behavior (hypothesis)

If a logged-out user can still view a protected page after pressing the browser Back button, the browser may be restoring cached page content even though the session has ended. Another possible cause is a frontend route guard that checks only local token presence without validating the session with the backend. I would reproduce the issue by logging in, opening a protected page, logging out, pressing Back, and attempting a protected action. I would inspect browser storage, network requests, route-guard logic, and API responses. The expected fix is to clear authentication data on logout, re-check authentication when protected routes load, and enforce authorization on the server for every protected endpoint. Sensitive content should not be available from cache after logout. I would add a regression test for logout followed by Back navigation. This analysis is a hypothesis until the behavior is reproduced.

4. Before submission

Execute the scenarios, record actual results, keep only reproduced defects, replace hypotheses with evidence, and attach screenshots. If a test passes, investigate an edge case instead of inventing a bug.